



Fundamental of programming (Using C++)

Heln1112

By Wondwossen Z. (MPH in Health Informatics and BSc. in Computer Science)



Chapter ONE

Introduction to Programming



Objectives



- In this chapter you will learn:
 - Computer fundamentals
 - Programming language levels
 - Programming paradigms
 - Algorithm design
 - Programming control structures
 - Software development life cycle



Computer fundamentals



- Computer is a machine that receives input, process and generate the desired output
- Has got two parts
 - HW- devices/ units
 - SW- instructions/ programs
- Computer is characterized by its accuracy, speed, storage capacity, versatility and diligence
- Though, limitation on intelligence and independence
- Known with its old history and generations
 - From a big, less performance and expensive machine to a portable, high performance and chip personal commodity

Computer Organization

➤ divided into six **logical units** or sections

➤ Input unit

- Text → keyboard
- Image → camera
- Sound → microphone
- Selection → mouse

➤ Memory unit:

- RAM- execution files
- ROM- BIOS

➤ Storage unit

- HD

➤ Output unit

- Display → Monitor
- Sound → Speaker

➤ Bus

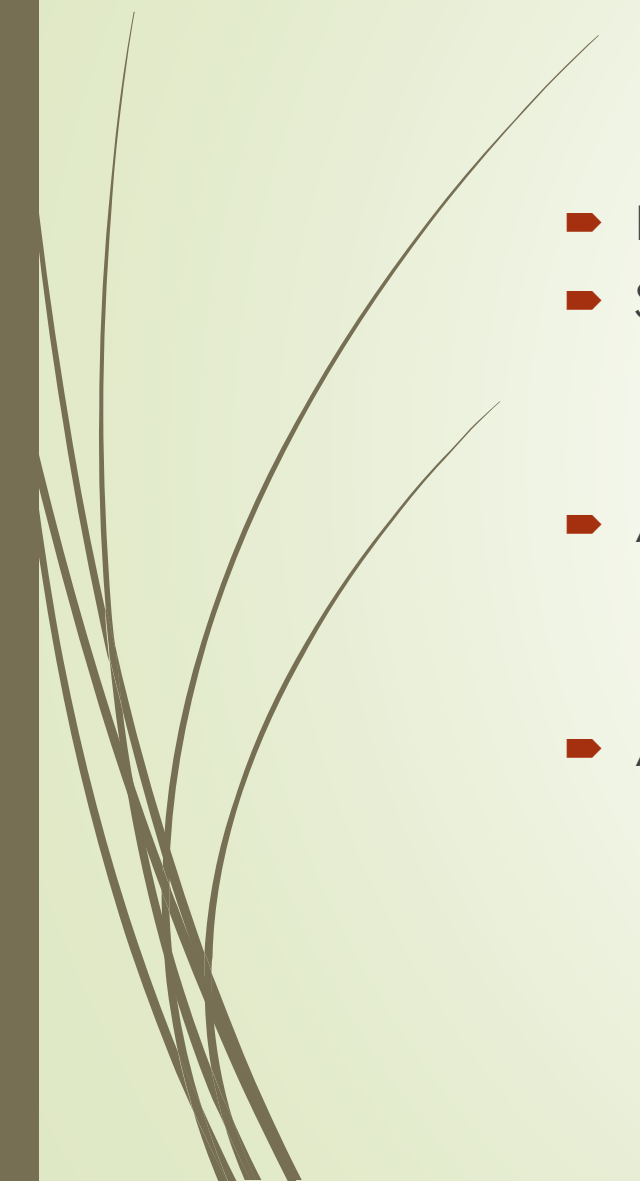
- Address
- Data

➤ CPU

- ALU → operation
- Control Unit → flow of data
- Register → operation files



Computer Instruction



- Divided in to two basic categories
- System software
 - Operating system
 - Utilities
- Application software
 - Standalone/ desktop based programs
 - Web based programs
- Applications can be developed for different purposes
 - Decision support
 - Entertainment..

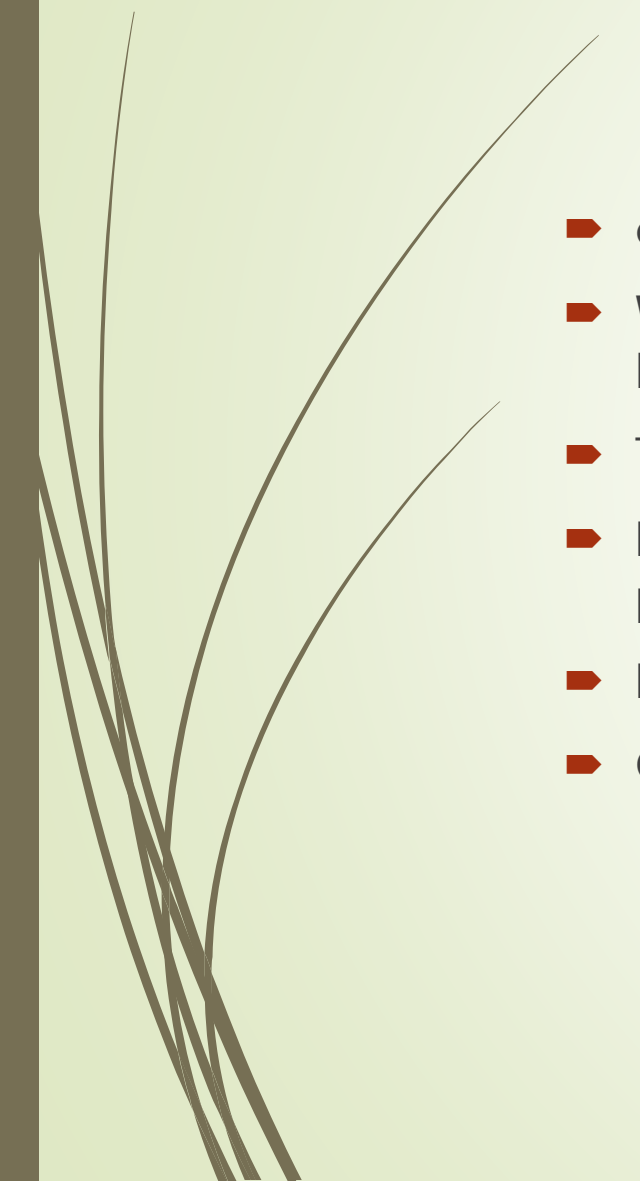


Personal, Distributed and Client/Server Computing

- Personal computing: every activity is controlled by individual users
- Distributed computing: branches/ local networks control activities
 - Standalone systems, shared disk
 - Multiple organizational users
- Client/server computing
 - Central Server provides all data and services
 - Client request data or service from the server
- C++ is used for writing software for OSs, for computer networking and for distributed client server applications..



The Internet and the World Wide Web

- 
- a global network of computers/ networks
 - **World Wide Web** allows computer users to locate and view multimedia-based documents
 - The Internet mixes computing and communication technologies
 - Internet-based computer applications can be developed by web programming languages
 - Internet improve accessibility of information, products or services
 - Creates one connected cyberspace



Programing language



- ▶ A program is a set of instructions that guide the computer in carrying out a specific task.
- ▶ A computer program consists of two essential parts.
 - ▶ The first one is description of actions that are to be performed.
 - ▶ Actions are described by statements
 - ▶ The second one is the description of data that are to be manipulated.
 - ▶ Data are described by definition and declaration.
- ▶ To write a program, we need to have programming language(software) such as c++ and java.
- ▶ Computer can not understand these languages, it has to be converted into machine language



Programing language translation



► Types of Translations:

- Assembler converts program developed with assembler Language to machine language
 - Compiler converts program developed with High Level Language to machine language
 - Interpreter converts program developed with High Level Language to machine language.
- Compiler converts all contents to machine code at a time where as interpreter converts the program to machine code line by line.



➤ Machine languages

- **object codes that are machine dependent**
- consist of strings of numbers (ultimately reduced to 1s and 0s)
- operations one at a time

Example:

+1300042774//load basepay

+1400593419//add overpay

+1200274027//store grosspay

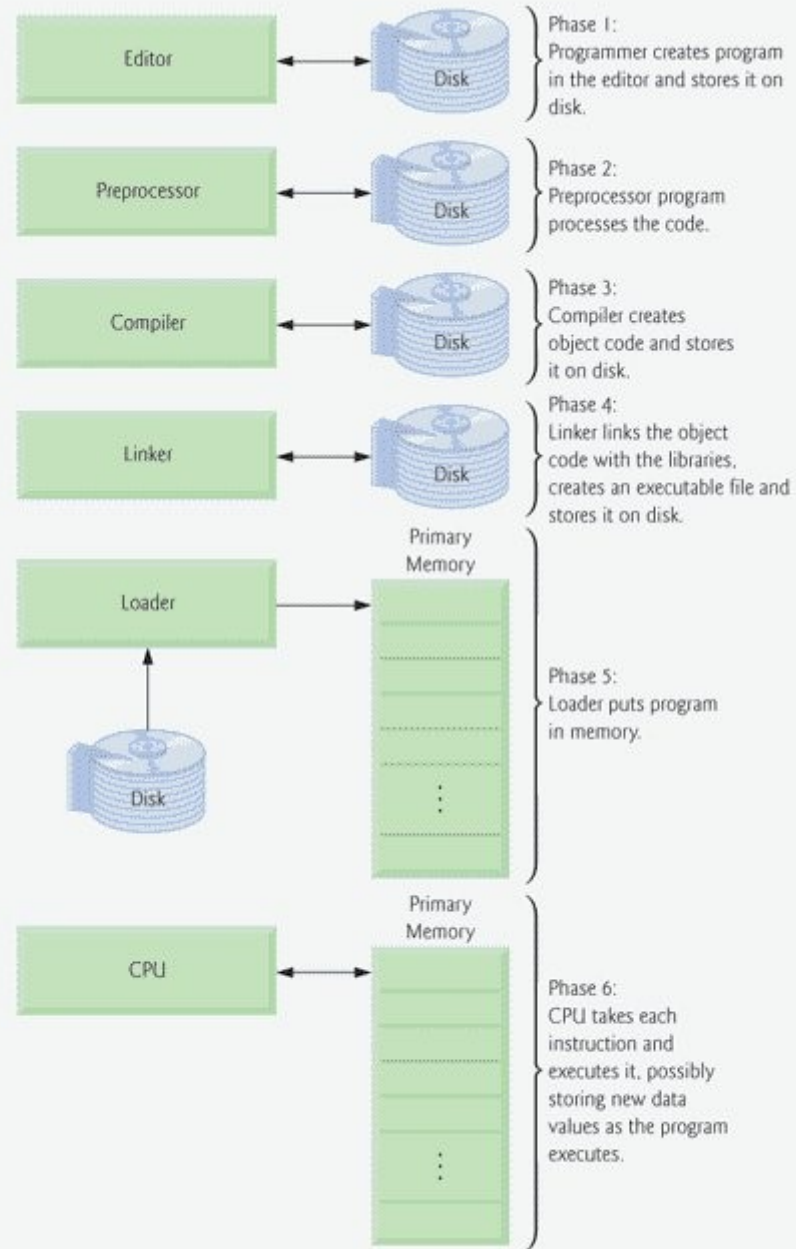
➤ Assembly languages

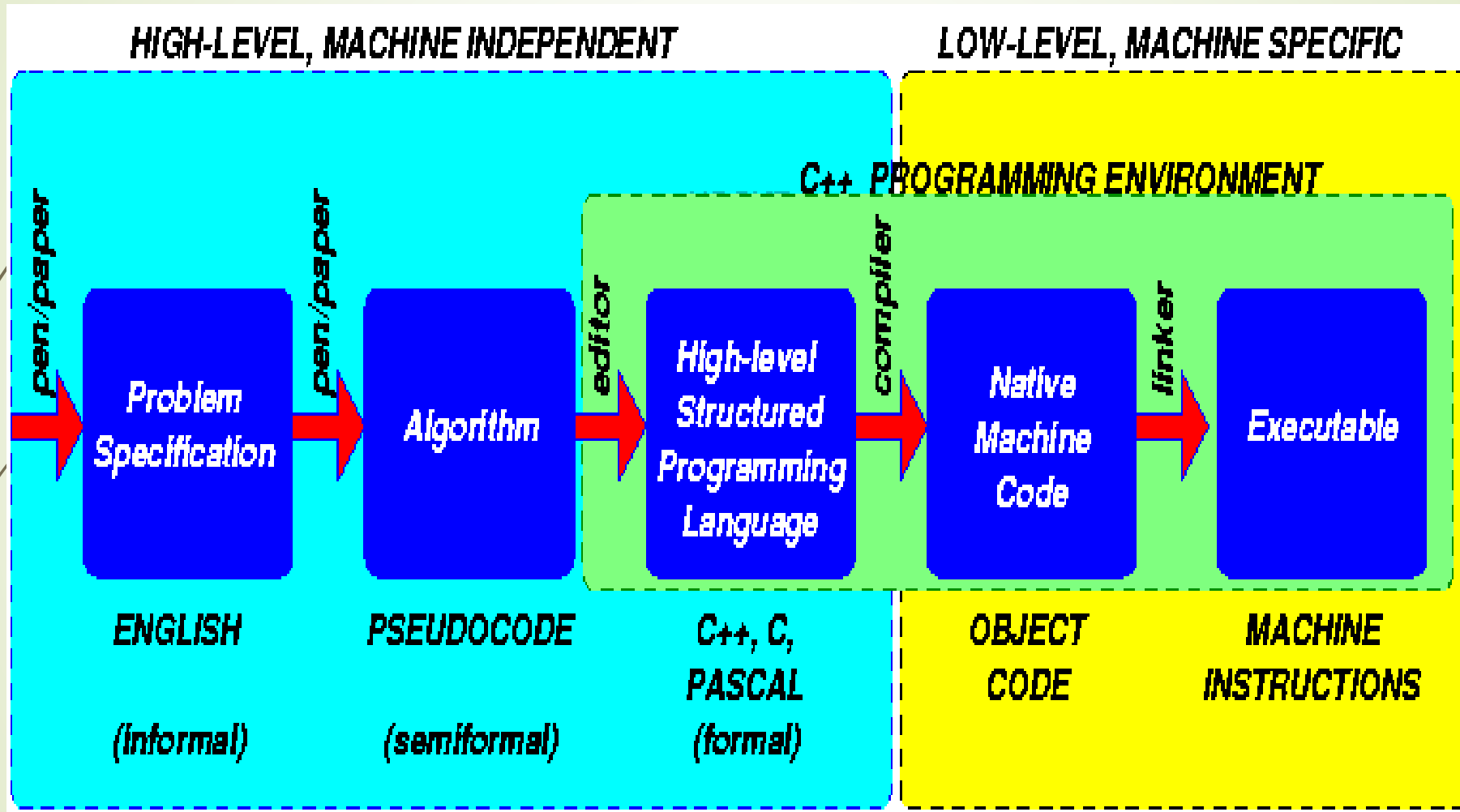
- using English-like abbreviations to represent elementary operations
- **Translator programs** called **assemblers** were developed to convert early assembly-language programs to machine language at computer speeds
- but programmers still had to use many instructions to accomplish even the simplest tasks
- Example:

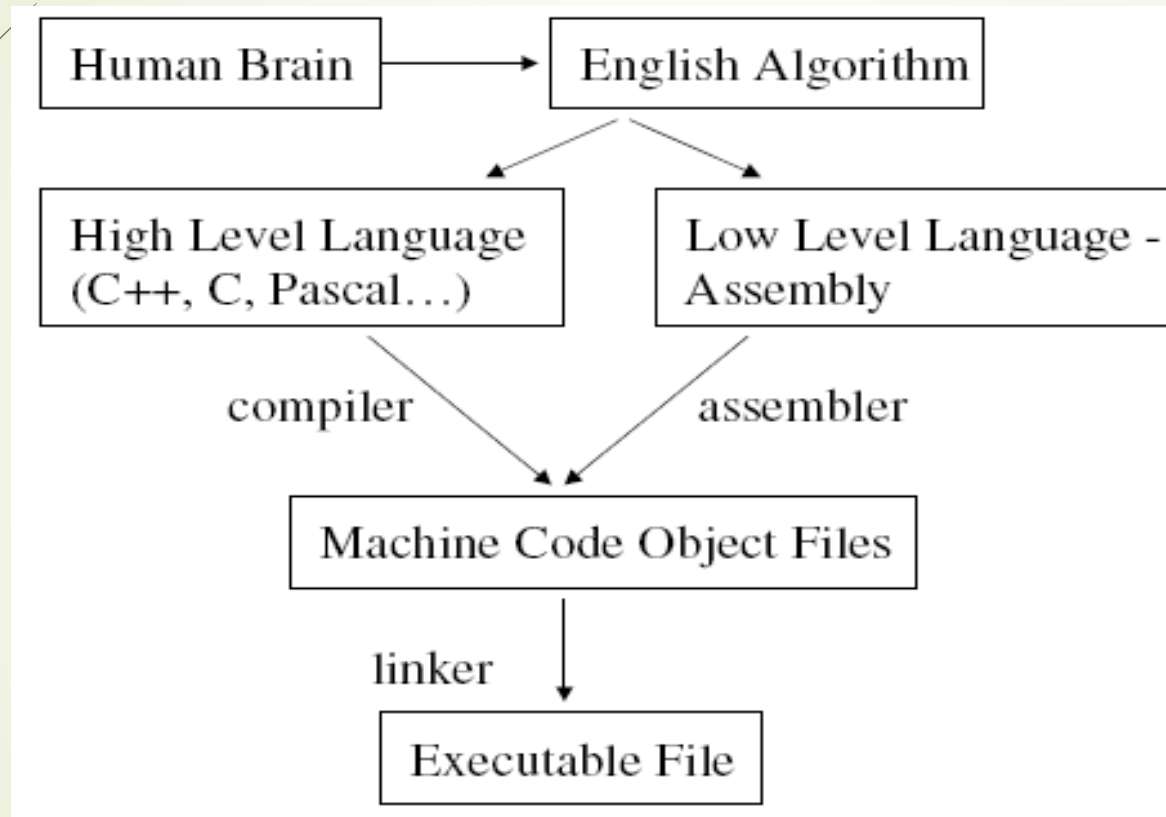
load	basepay
add	overpay
store	grosspay

➤ High-level languages

- To speed the programming process, **high-level languages** were developed in which single statements could be written to accomplish substantial tasks
- Translator programs called **compilers** convert high-level language programs into machine language
- High-level languages allow programmers to write instructions that look almost like everyday English and contain commonly used mathematical notations





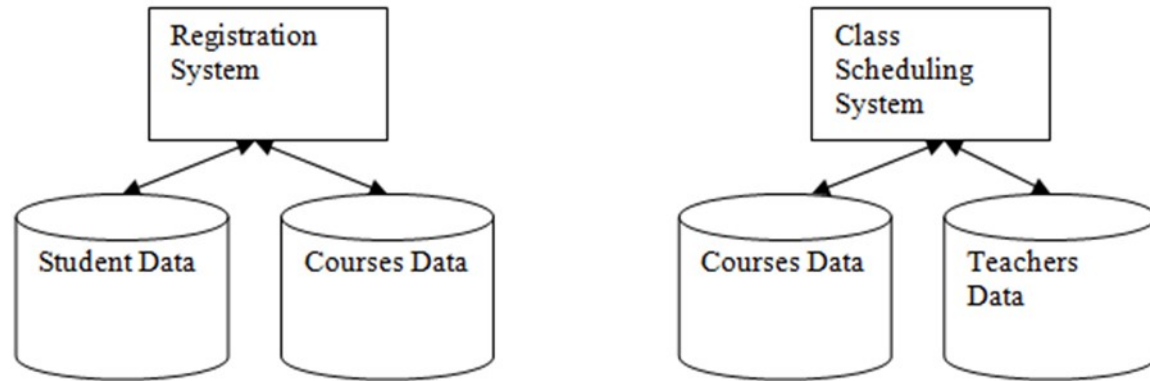




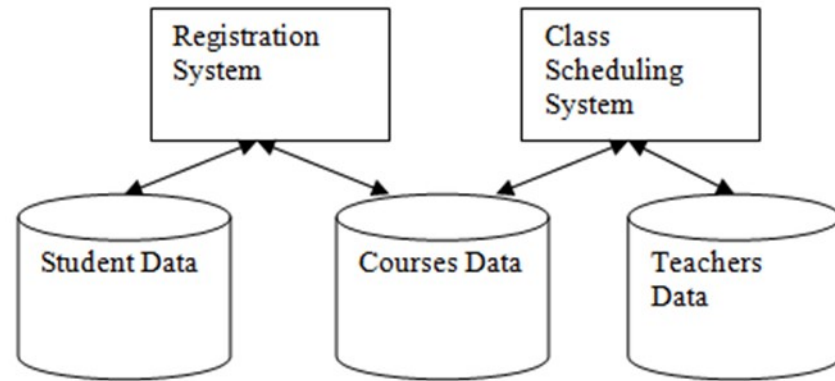
Programming paradigms

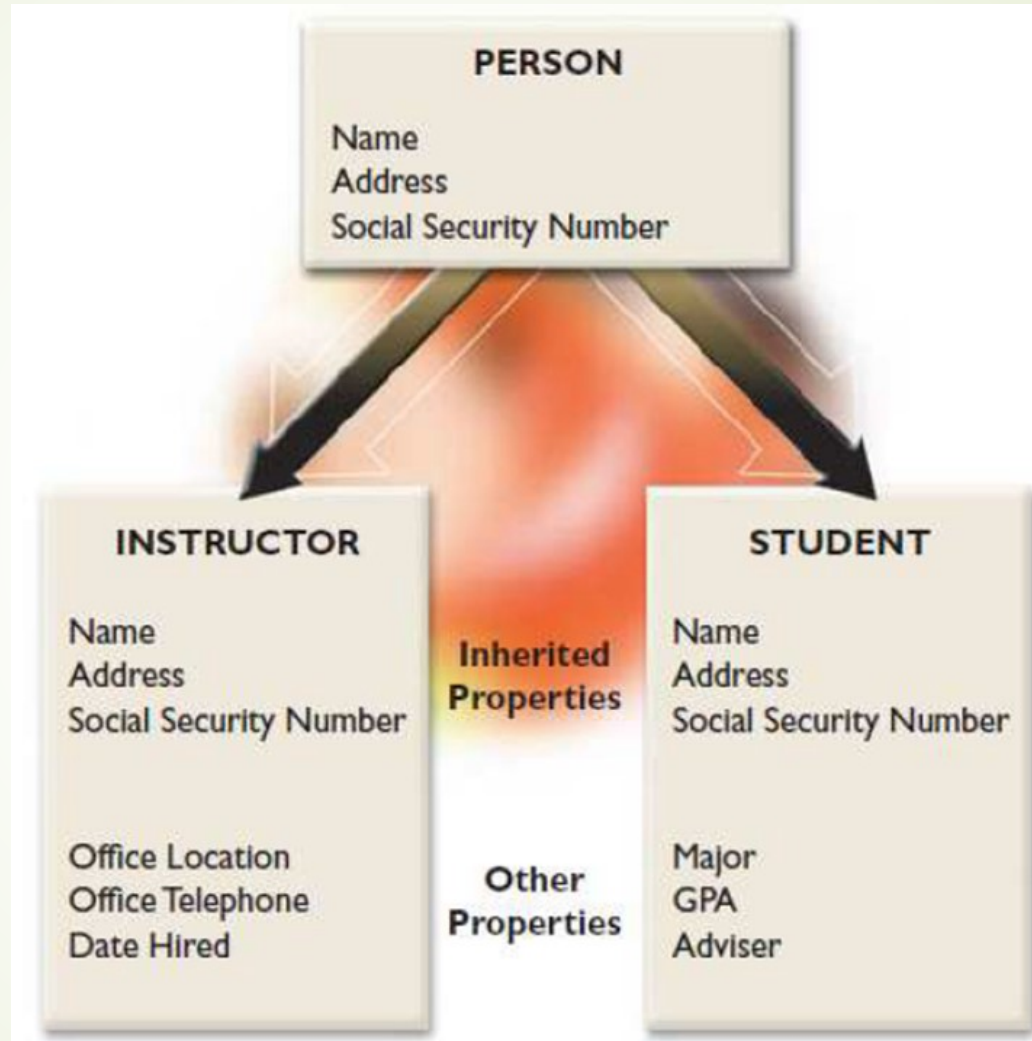
- The approach to design a solution to a programmable problem
 - Procedural
 - Structured
 - Object oriented
- Procedural/ modular approach- it is a process oriented approach in which separated solution will be designed to a subsystem/ module → Modules
- Structured approach- it is a data oriented approach targeted to minimize duplication of data among subsystems → DFDs
- Object oriented approach- it categorizes members of the system having common states and behaviors as objects

Process-oriented approach



Data-oriented approach







Algorithm



- An algorithm is a sequence of a **finite** number of steps arranged in a specific logical order which, when executed, produces the solution for a problem.
- An algorithm must satisfy these requirements:
 - It must have an input
 - It must have an output
 - It should not be ambiguous (there should not be different interpretations to it)
 - It must be general (it can be used for different inputs)
 - It must be correct and it must solve the problem for which it is designed
 - It must execute and terminate in a finite amount of time
 - It must be efficient enough so that it can solve the intended problem using the resource currently available on the computer

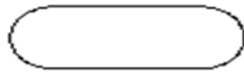


Pseudo code

- Pseudo code is Structured English used to represent an algorithm.
- Pseudo code is English-like language used to express algorithms (less restrictive/formal than a programming language) .
- It is easy for both users and programmers to read and write algorithms regardless of the programming experience.
- There is no standard pseudo code but we should follow some conventional rules below:
 - Statements are written in simple English.
 - Each statement is written on a separate line.

Flowchart

- A flowchart is a graphical diagram that depicts the “flow” of a program using a series of standard geometric symbols and lines, which are connected according to the logic of the algorithm. There are a lot of symbols used in flowcharting but the common five are:



Terminal symbol

The terminal symbol indicates the starting or stopping point in the program. Every flowchart must begin and end with this symbol.



Process symbol

This process symbol represents any single process in an algorithm, such as assigning the value to a variable or performing a calculation, etc



Input/output symbol

Input/output symbol represents the process of getting input and returning the output



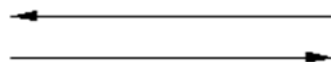
Decision symbol

The decision symbol represents a decision in the algorithm. Alternative paths are followed depending on whether the condition is true or false



Connector symbol

The connector symbol is used to join the flowlines in a flowchart



Flowlines

Flowlines with arrowhead connect various symbols in a flowchart.

Programming control structures

- ▶ Programming control structure controls how the each statement (step) of the algorithm flows (executes). Control structures in most programming languages typically include the following.

1. Sequence

- ▶ The sequence control structure is the straightforward execution of one processing step after another.

- ▶ **Sequence's pseudo code:**

Program Start

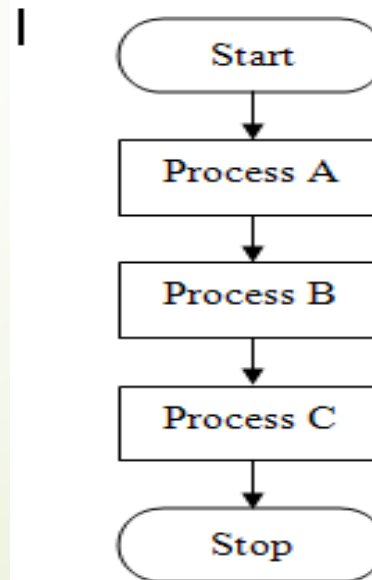
Process A

Process B

Process C

Program Stop

Sequence's flowchart:





2. Selection

- The selection control structure is the presentation of a condition and two actions between two actions. The choice depending on whether the condition is true or false.

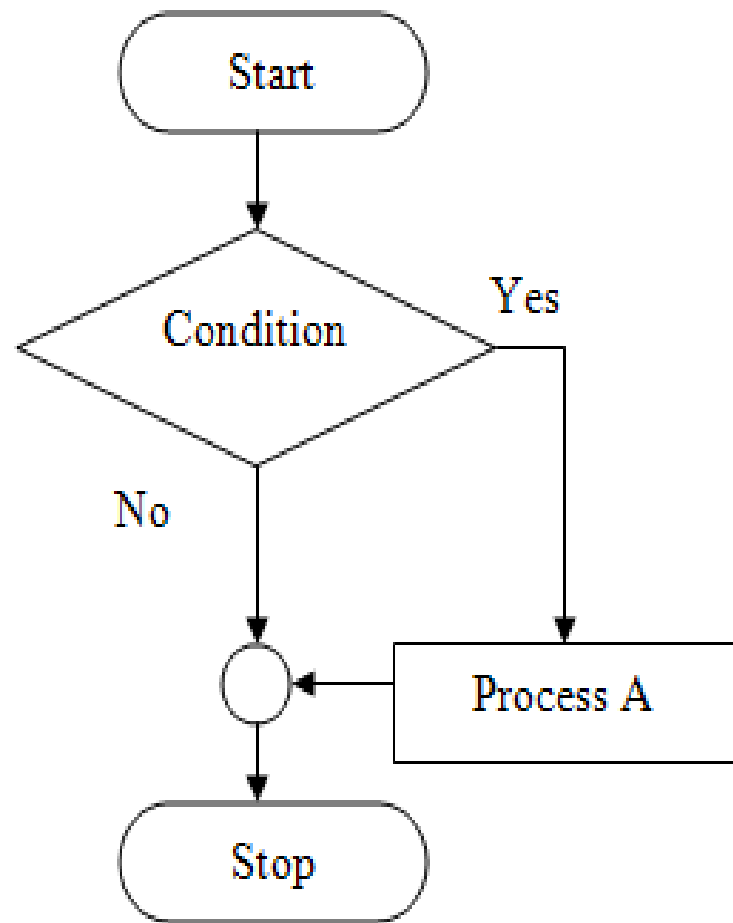
- **IF' s pseudo code:**

- Program Start
- IF (Condition) Then
- Process A
- End IF
- Program Stop

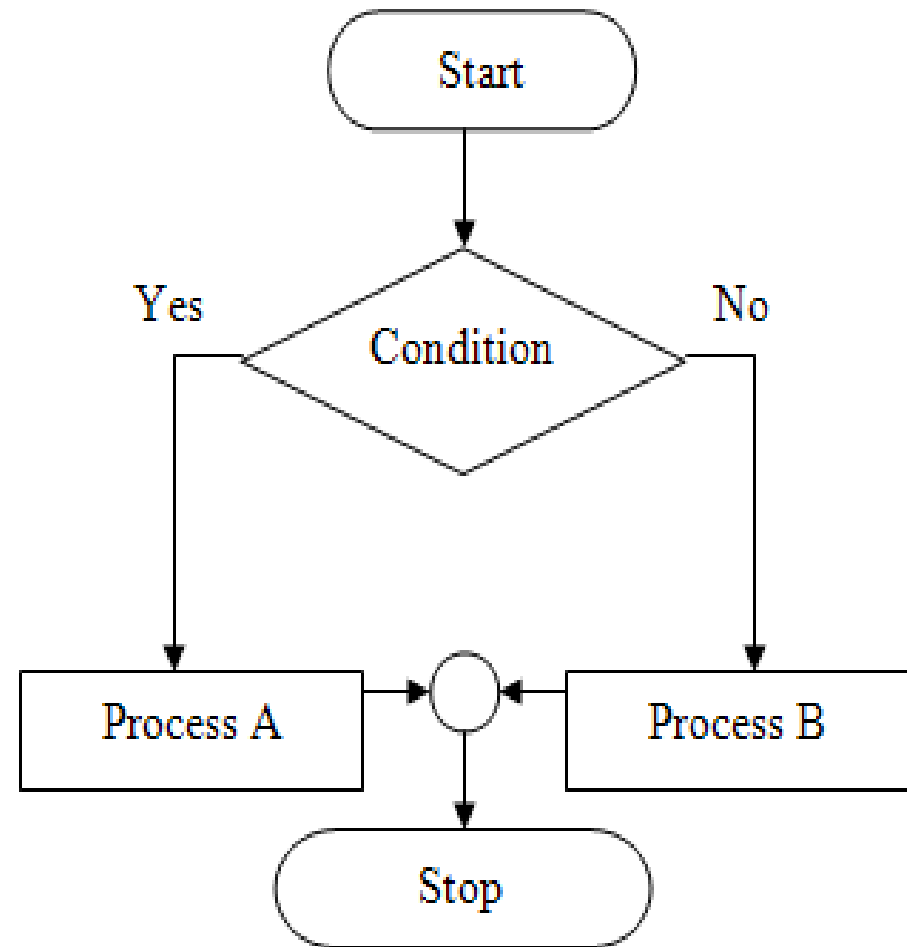
- **IF-ELSE' s pseudo code:**

- Program Start
- IF (Condition) Then
- Process A
- ELSE
- Process B
- End IF
- Program Stop

IF's flowchart:



IF-ELSE's flowchart:



3. Iteration (Looping)

- the representation of a set of instructions to be performed repeatedly, as long as the condition is satisfied (TRUE or FALSE). There are two main types of iteration: DOWHILE and REPEAT_UNTIL.

- DOWHILE's pseudo code:**

Program Start

DOWHILE (Condition)

Process A

Process B

Process C

Process D

Process E

Process F

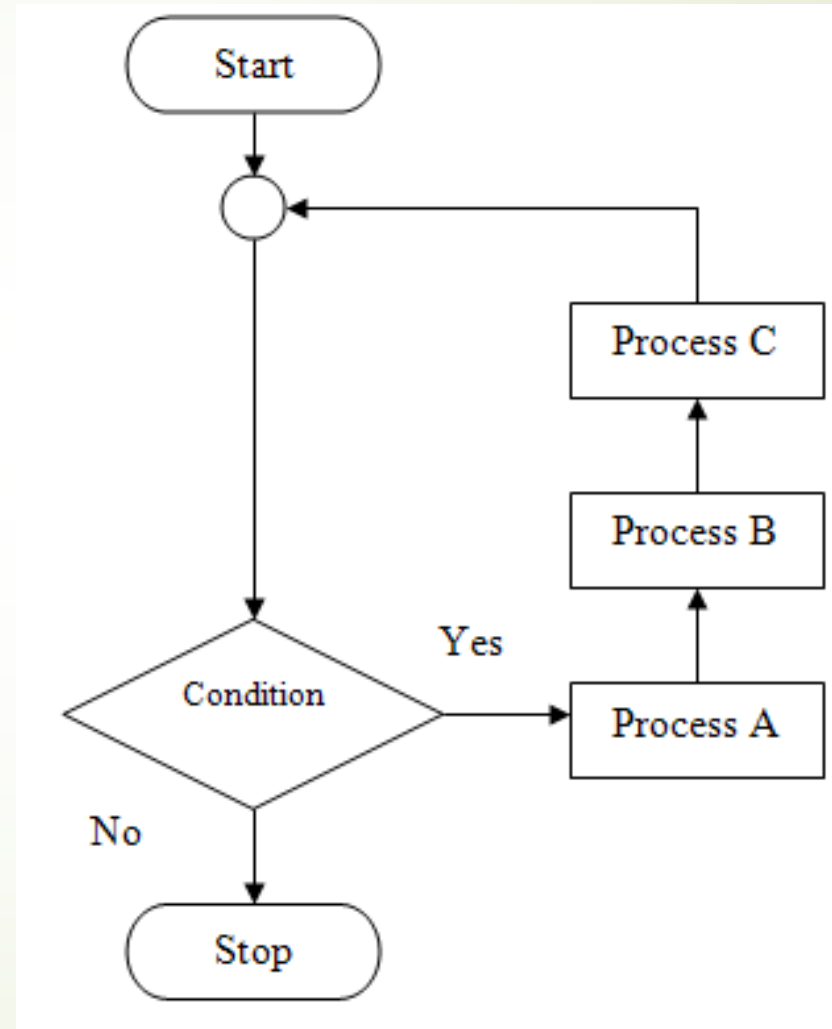
.

.

Process n

ENDWHILE

Program Stop



➤ REPEAT_UNTIL The loop will be terminated if the result of condition is **True**.

➤ **REPEAT UNTIL's pseudo code:**

Program Start

REPEAT

Process A

Process B

Process C

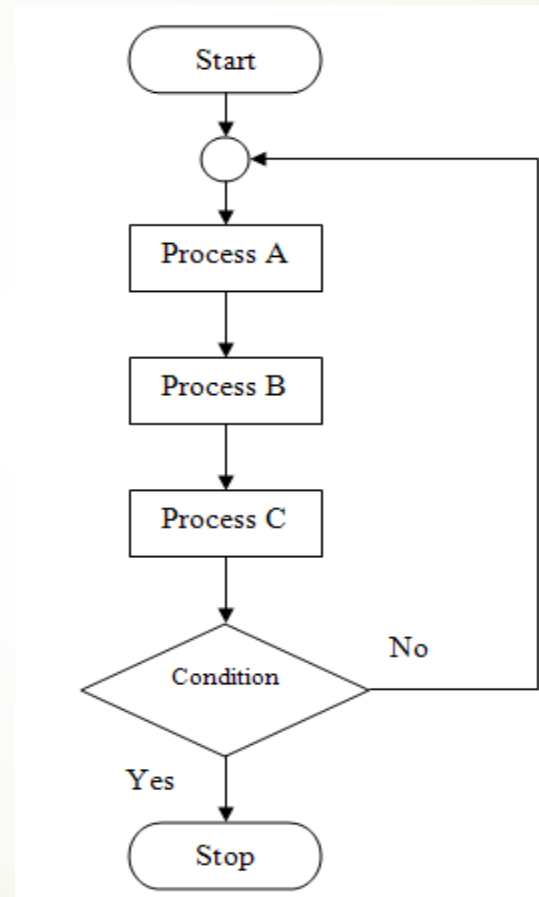
.

.

Process n

UNTIL (Condition)

Program Stop





Programming control structures

- **Algorithm development examples**

- In developing an algorithm, the problem should be defined and divided into three separate components:

- Input: a list of the source data provided to the problem.
- Output: a list of the outputs required.
- Processing: a list of actions needed to produce the required output.

- **Example01: Add three numbers**

- Identify I/O and processes
 - Input: number1, number2, number3
 - Output: Total
 - Process: read three numbers, add numbers together, print total to the screen

➤ **Pseudo code:**

Program Start

Set number1, number2, number3, total = 0

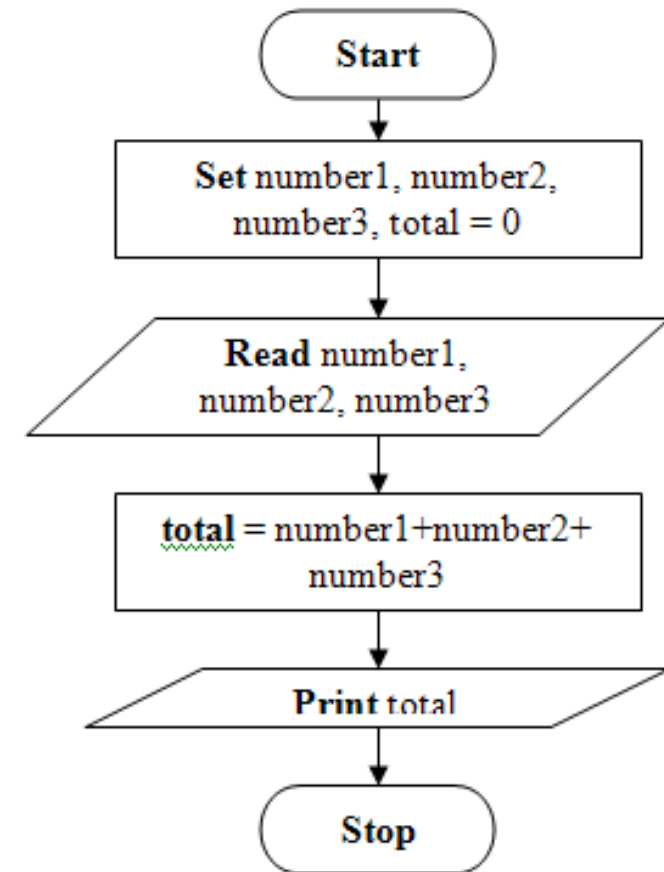
Read number1, number2, number3

total = number1+number2+number3

Print total

Program Stop

Flowchart:



- **Compare two numbers**

- Identify I/O

- Input: number1, number2

- Output: larger number

- Process: Read number1, number2, Compare number1 and number2
Print output message on the screen

➤ **Pseudo code:**

Program Start

Set number1, number2 = 0

Read number1, number2

IF number1 > number2 **THEN**

Print "Number1 is greater than number2"

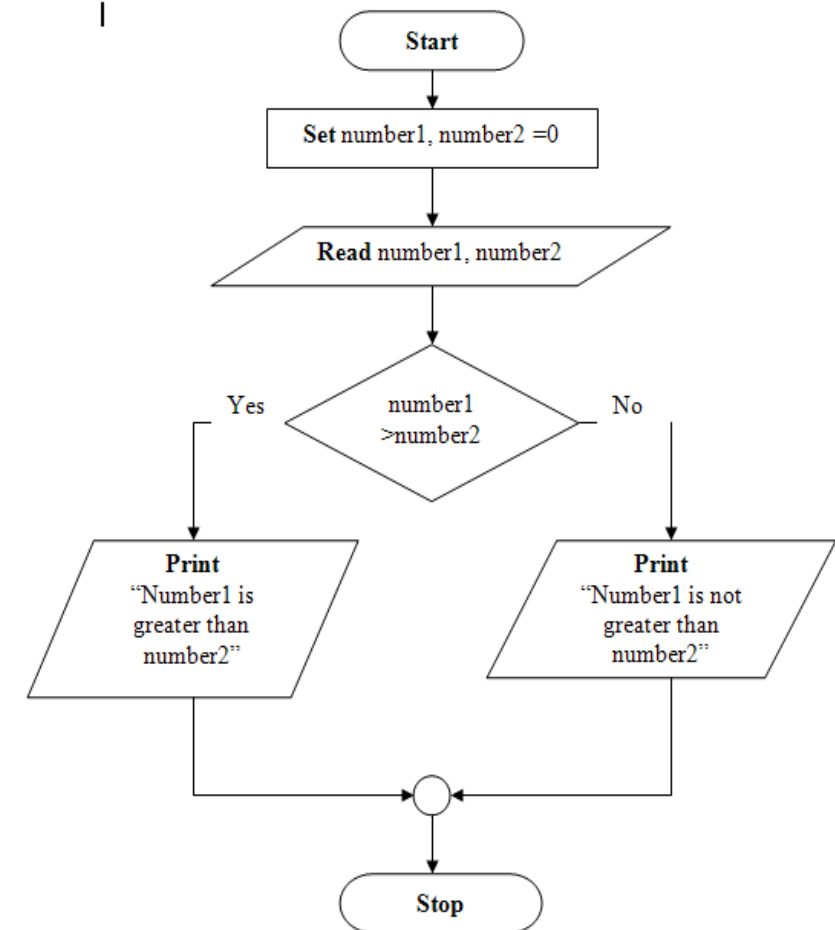
ELSE

Print "Number2 is greater than number1"

ENDIF

Program Stop

Flowchart:



- **Find average score**

- Input: Score (10 scores)

- Output: Average

- Process: Read score, Calculate total,
Calculate count Compute average
Print average on the screen

► **Pseudo code:**

Program Start

Set score, total, count, average = 0

DOWHILE count<10

Read score

Add score to total

Add 1 to count

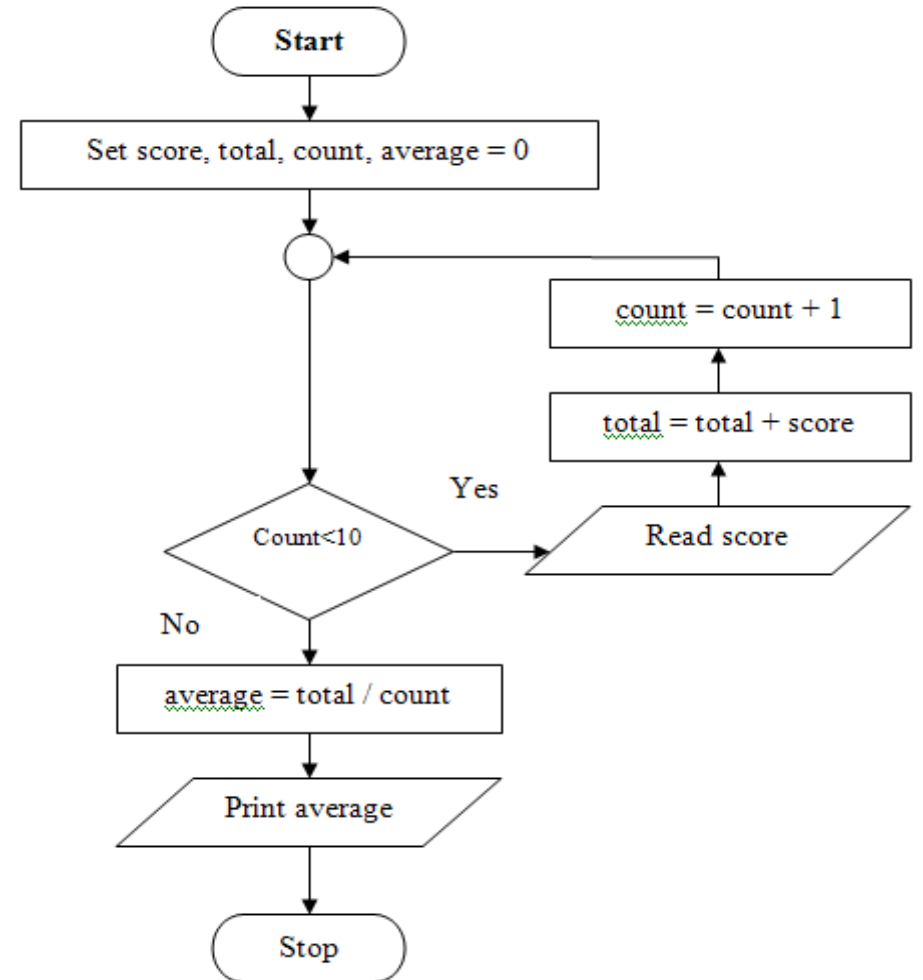
ENDDO

average = total/count

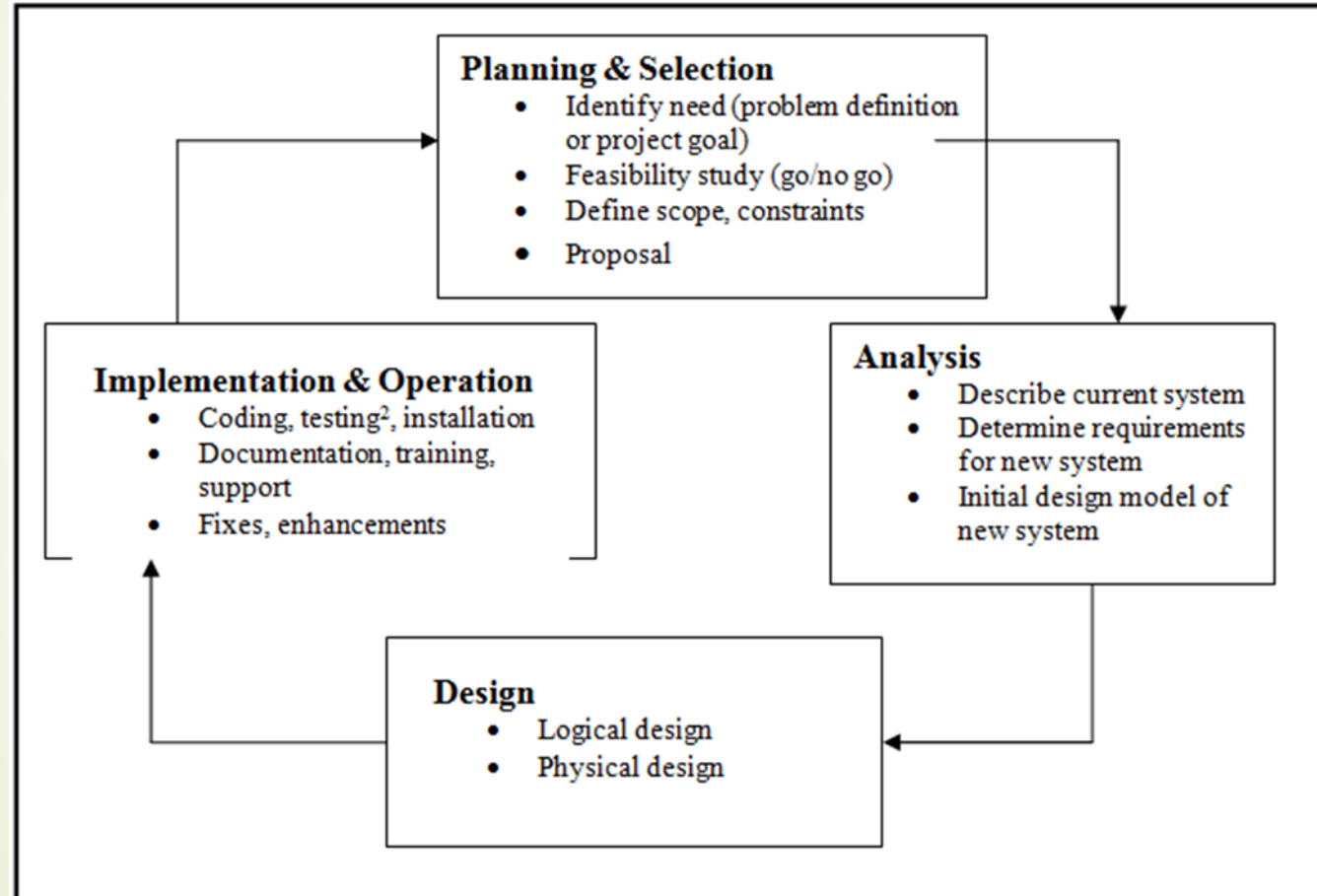
Print average

Program Stop

Flowchart:



System Development life cycle (SDLC)





Assignment

- Write a mini software project proposal in a group
- 